Logotipo UNAM pendiente

Logotipo FI pendiente

Universidad Nacional Autónoma de México
Facultad de Ingeniería

Estadio de fútbol en OpenGL

Manual de usuario y manual técnico — boceto

Asignatura:	Computación Gráfica (confirmar nombre oficial)
Equipo:	**Pendiente: nombres completos en orden alfabético**
Profesor(a):	**Pendiente: nombre**
Fecha de entrega:	**Pendiente: confirmar con el profesor**

Versión preliminar basada en `graficaLaboratorio/Configinicial/`.

Índice

Parte I

Manual de usuario

1. Propósito y alcance

El programa muestra un estadio pequeño de fútbol sin techo, con cancha, gradas, fachada y cámaras de transmisión. Permite recorrer la escena y activar un penal, fuegos artificiales y, cuando se cargan sus recursos, un autobús. Los resultados del penal se muestran en el título de la ventana y en la consola. La clasificación de objetos y animaciones para la evaluación sigue pendiente de validación.

2. Preparación y ejecución

En Linux se requieren un compilador de C/C++, `pkg-config` y los paquetes de desarrollo de GLFW, GLEW, GLM y Assimp. Conservar la carpeta `Configinicial/` junto a `Shader/`, `Models/` y `SOIL2/`; ejecutar desde ella para que funcionen las rutas relativas:

```
cd graficaLaboratorio/Configinicial
./build.sh
./graficaLaboratorio
```

El script usa `gcc` y `g++` para crear `graficaLaboratorio`. La vista inicial es la cámara de transmisión 4; para recorrer libremente el estadio pulsar `0`.

3. Controles

Tecla	Acción comprobada en `main.cpp`
`0`	Cámara libre: mover con W/A/S/D o flechas y orientar con el ratón.
`1-2`	Vista desde drones; W/A/S/D o flechas desplazan el dron, Q/E cambian altura y Shift acelera.
`3-8`	Vistas fijas de transmisión.
`9`	Vista exterior de la fachada.
`N`	Iniciar un penal si el anterior terminó; el portero reacciona durante el tiro.
`P`	Lanzar fuegos artificiales.
`R`	Reiniciar penal, balón y contadores.
`B`	Iniciar o pausar autobús *solo si sus archivos se cargaron correctamente*.
`Espacio`	Alternar la animación de color de la luz puntual.
`[ / ]`	Cambiar la orientación inicial de los modelos de cámara en pasos de 90 grados.
`T/G, Y/H, U/J`	Mover la luz puntual en X, Y y Z.
`Esc`	Cerrar la ventana.

Nota: el movimiento W/A/S/D en vistas 3–9 no cambia la cámara; los ajustes T/G, Y/H, U/J funcionan en modo libre.

4. Prueba guiada y problemas conocidos

Pulsar `9` para ver la fachada, `0` para caminar por el estadio, `N` para el penal y `P` para los fuegos. Cambiar a las vistas 1 y 2 para desplazar drones; observar también el ave y el trofeo. **Pendiente:** añadir capturas propias con la vista y la animación identificadas.

Si falla la compilación, revisar dependencias con `pkg-config` y conservar el directorio de trabajo indicado. Si no aparecen texturas o cámaras, comprobar `Shader/` y `Models/Camaras/`; las rutas se interpretan desde `Configinicial/`. En Linux el autobús no carga en el estado revisado: `main.cpp` usa `Models/Autobus/`, pero la carpeta del repositorio es `Models/autobus/`. Por ello `B` no tiene efecto hasta corregir la diferencia de mayúsculas.

Parte II

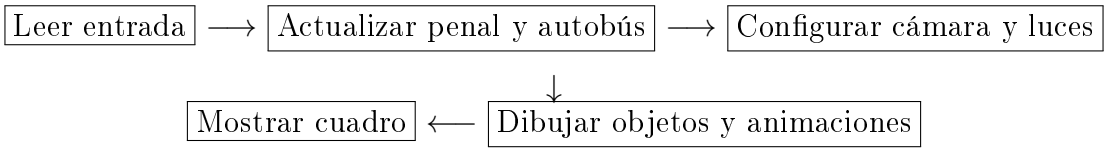
Manual técnico

5. Objetivos y alcance

Recrear por código un estadio abierto y animaciones relacionadas con el fútbol mediante OpenGL, conservando como base la estructura de cámara, shaders e iluminación de las prácticas. La fachada, gradas, cancha, personajes y otros objetos se forman principalmente con funciones de dibujo; los modelos de las cámaras se importan como OBJ. **Pendiente:** incorporar imágenes de referencia de fachada y espacio y confirmar con el profesor qué objetos cuentan en el mínimo de siete.

6. Arquitectura y flujo del programa

`main.cpp` inicia GLFW/GLEW, construye VAO/VBO, shaders, texturas y modelos; registra los callbacks de teclado y ratón. Cada cuadro sigue este flujo:



Pendiente: sustituir este esquema preliminar por un diagrama de flujo final, con decisiones y estados del penal, revisado por el equipo.

7. Módulos y recursos

Archivo o grupo	Responsabilidad
`main.cpp`	Inicialización, controles, escena, estados de penal, autobús y renderizado.
`Camera.h`, `Shader.h`	Transformaciones de vista y carga/compilación de shaders (base docente de las Prácticas 2, 7 y 8).
`Shader/lighting.vs`, `lighting.frag`	Shaders de iluminación Phong/Blinn-Phong extendidos desde la Práctica 7 con proyecciones procedurales y múltiples fuentes.
`Model.h`, `Mesh.h`	Carga y dibujado de los modelos OBJ importados con Assimp (base de las Prácticas 5 y 8).
`Models/Textures/`	Mapas de color PBR CC0 y atlas procedurales.
`Models/Camaras/`	Cámara separada en base/cabezal, materiales e imagen del logo Liga MX.

7.1. Procedencia y correspondencia con el código base docente (Prácticas 1 a 10)

El proyecto integra de manera formal la arquitectura docente construida a lo largo de las sesiones de laboratorio de la asignatura:

- **Práctica 1 (Dibujo 2D e infraestructura):** Inicialización del contexto GLFW, enlaces a funciones OpenGL vía GLEW y ciclo continuo de renderizado con intercambio de búferes (`glfwSwapBuffers`) y despacho de eventos.
- **Prácticas 2 y 3 (Proyecciones y Modelado geométrico):** Sistema de coordenadas y álgebra lineal con GLM (`glm::perspective`, `glm::lookAt`, matrices de transformación) y definición de primitivas volumétricas mediante Vertex Buffer Objects (VBO) y Vertex Array Objects (VAO) para cubos y prismas rectangulares.
- **Práctica 4 (Modelado jerárquico):** Estructuración de transformaciones subordinadas en cascada (`modelTemp`) para la animación articulada de extremidades superiores e inferiores de jugadores y cuerpo arbitral.
- **Prácticas 5 y 8 (Carga de modelos y Assimp):** Estructuras `Mesh.h` y `Model.h` para la lectura de geometría, índices y materiales desde archivos OBJ y MTL externos.
- **Práctica 6 (Texturizado):** Gestión de imágenes mediante `SOIL2` y `stb_image`, configuración de parámetros de muestreo (`glTexParameteri`), filtros lineales/vecino más cercano y habilitación de atlas de textura en shaders.
- **Prácticas 7 y 8 (Iluminación y Materiales):** Implementación de `Camera.h` con vectores directores y ángulos de Euler, junto con los shaders `lighting.vs` y `lighting.frag` que resuelven el modelo Phong/Blinn-Phong dividiendo la contribución en componentes ambiental, difusa y especular, integrando luz solar direccional, fuentes puntuales y reflectores tipo *spotlight*.
- **Prácticas 9 y 10 (Animaciones y Máquinas de estados):** Dinámicas temporales basadas en máquinas de estados finitos y cuadros clave (*keyframes*) para secuencias complejas y cinemática en la escena.

El programa genera también texturas de un píxel para los elementos de color sólido. La playera del tirador usa un atlas original azul marino y dorado (frente y espalda), con UV explícitas en un VAO de torso independiente; el cubo compartido permanece intacto. En el shader, la ruta de cámara conserva los colores importados; los mapas procedurales de la escena son de color, sin relieve geométrico. **Pendiente:** inventariar licencias, créditos, referencias visuales y relación exacta de estos archivos con el código del profesor. Las torres de iluminación usan el mapa difuso 1K *Worn Shutter* de Poly Haven (Dimitrios Savva, CC0: `https://polyhaven.com/a/worn_shutter`); los paneles luminosos siguen separados. Las cabezas del tirador y el portero comparten un atlas pixel art de seis caras con UV explícitas y filtrado por vecino más cercano. El generador `generate_player_head.py` incluye otras expresiones, pero la escena usa únicamente el atlas neutral; no se animan los rostros.

8. Animaciones implementadas y evaluación pendiente

- Sencillas candidatas: ave en órbita con aleteo (`drawBird`), rotores de dron (`drawDrone`) y giro de trofeo (`drawTrophy`). El cabezal de las cámaras sigue al balón.
- Complejas candidatas: penal (`Animation`, estados, trayectoria con altura variable y reacción del portero) y fuegos artificiales (`drawFireworks`, estallidos con partículas calculadas por código).
- Autobús: `updateBusAnimation` define trayecto, giro y ruedas, pero la ruta de recursos incorrecta impide usarlo en Linux actualmente.

Pendiente: capturar cada una en ejecución y confirmar con el profesor que cumplen exactamente tres sencillas y dos complejas; la existencia del código no equivale a la aceptación de la rúbrica.

9. Metodología, cronograma y validación

El trabajo se ha desarrollado por incrementos: geometría por código, animación, selección de mapas de color, corrección de UV del logo y pruebas locales con `build.sh`. Existe `tests/check_camera_logo.py` para verificar UV y ruta de textura. **Pendiente:** completar el siguiente Gantt con fechas reales, responsables y evidencias; no se han supuesto fechas.

Actividad	Inicio	Fin	Responsable
Escena y fachada	Por llenar	Por llenar	Por llenar
Animaciones y modelos	Por llenar	Por llenar	Por llenar
Pruebas y documentación	Por llenar	Por llenar	Por llenar
Video y entrega	Por llenar	Por llenar	Por llenar

Pendiente: compilar y probar en una copia limpia y en el equipo de entrega, medir los tamaños por objeto, verificar los siete candidatos texturizados y registrar resultados de las cinco animaciones.

10. Limitaciones y trabajo pendiente

Los asientos, el uniforme del portero, el balón, el ave y otros elementos todavía usan texturas de color plano; las cabezas de ambos jugadores, la playera del tirador y las cámaras sí usan mapas de imagen. El autobús queda deshabilitado por la diferencia entre `Autobus` y `autobus` en Linux. No hay evidencia incorporada aquí de aprobación del profesor, presentación, video ni entrega en Classroom. Véase `PENDIENTES_PROYECTO.md` para el estado detallado.

11. Conclusiones y referencias

Pendiente: una conclusión individual por integrante, conclusiones generales basadas en pruebas observadas y nombres completos del equipo. No atribuir resultados no verificados.

Referencias por completar: consigna del proyecto final y prácticas proporcionadas por el profesor; documentación oficial de GLFW, OpenGL y Assimp. **Pendiente:** añadir autor, URL y licencia de cada imagen de referencia y mapa de textura utilizado.